

Regex in JavaScript

Regular expressions are supported in JavaScript, and can be quite useful in ensuring that the data entered by the user is in the correct format

Grep on Windows

grep can be installed on Windows as a part of Cygwin or MSYS. A standalone Windows port is also available.

Regular Expressions

“Regular Expressions” are a way of describing patterns of text that can be useful for processing text documents or when looking for a pattern and possibly replacing it with another.

Kshitij Sobti

kshitij.sobti@thinkdigit.com

Imagine you have a rather long document with a single repetitive misspelling. Imagine a Mr. Verma is displeased that his surname has been misspelled as “Varma”. It’s simple enough to remedy; even a text editor such as notepad can perform a search and replace operation for something as simple as this.

However, if we start making it even a little more complicated, say we’re searching for a pattern instead of a misspelling, such simple measures start to fail. Imagine having to replace every occurrence of a pattern of URLs with another pattern. For example, if due to restructuring on a website, URLs that had the pattern:

```
http://www.website.com/[day]/[month]/[year]/article.html
```

have been changed to:

```
http://www.website.com/[year]/[month]/[day]/article.html
```

What can one do? Searching and replacing won’t do the trick here, and unless you are dealing with very few URLs, this is just too much to do manually. Alternatively, imagine wanting to look for a sequence of two words, while being able to count the number of times they occur in some text, the catch here being that these two words could be separated by any kind of whitespace, a tab, a space, a line break, etc. These situations are easily handled by Regular Expressions.

What are Regular Expressions?

Regular Expressions (Regex) are a way of defining a pattern to be extracted / replaced / processed in a body of text. Many programming languages support regular expressions, either as part of that language or as part of a library.

The exact syntax and usage of Regex in different programs differs, however, the basic principles remain the same. Many applications, such as Notepad++, TextPad and even LibreOffice / OpenOffice include support for regular expressions. Linux users might be aware of the grep command, which is a Regex search engine tool for your text files or input. The Linux grep command is powerful enough to let you search through the files on your system digesting data, looking for the pattern you provide.

The Basics

Since it is patterns that we need to match rather than text sequences, regular expressions give special meanings to standard characters so that they can match multiple sequences.

For example, if we want to match any double-digit number, we do that using the regular expression “\d\d”

Here \d stands for any numeric digit.

What if we had to match any double-digit even number though? Here we could use \d[02468]

Here the contents inside the square brackets define a character set, and will match any enclosed character. This also means that we could have used

[0123456789] instead of \d, but there’s an even better way, by using these ranges [1-4][3-7]

The above will match any double-digit number that begins with 1, 2, 3 or 4, and has 3, 4, 5, 6, or 7 as the second digit. These kinds of ranges can be used even with alphabets; [a-z] matches any small-case alphabet.

Similarly, it’s possible to invert a character set; [^1-5] would mean any character other than the ones between 1 and 5. Note that any character other than 1 to 5 would include alphabets as well!

The pipe character “|” can be used to provide alternatives. For example if you’re looking for occurrences of both “June” and “July” you could use Ju(ne|ly)

For convenience sake, most regular expression implementations have easy-to-use character classes, such as the previously mentioned “\d” for matching any digit. One of the most important ones is the period character or “.” that matches any character other than a newline. Refer to the table Regex reference provided for your help.

Matching repetitions

So far, we’ve looked for double-digit numbers; what about patterns of arbitrary length? For those too, Regex has a way. If we need to match any number, i.e. any uninterrupted sequence of digits, we can use

\d+

The “\d” matches any digit, and the plus sign (+) signifies that the previous expression should occur once or more. Decimal numbers are a more interesting example. Decimals can have any number of digits before the decimal point (including no number, as .23), and at least one number after the decimal point. This could be matched with \d*\.\d+

This can be divided into three parts:

- \d* – Here the * matches zero or more occurrences of the previous expression i.e. \d or any numeric character.
- \. – We need to match a period / dot character, however that already has a different purpose, which is to match any character. To match the period / dot character itself, we need to use it with the backslash.